

Couple Sort

Indradeep Hayaran

Student, Computer Science & Engineering,
PDPM Indian Institute of Information Technology,
Design and Manufacturing, Jabalpur, M.P., India.
indradeephayaran@iiitdmj.ac.in

Pritee Khanna

Associate Professor, Computer Science & Engineering,
PDPM Indian Institute of Information Technology,
Design and Manufacturing, Jabalpur, M.P., India.
pkhanna@iiitdmj.ac.in

Abstract—Computer systems and applications deal with numerous cases which require rearrangement of certain entities either in ascending or descending order. The operation required to perform this task is called Sorting. The sole purpose of designing new algorithms is to reduce the number of computations and time taken to perform a particular sort. Also there are cases where an algorithm is best suitable than the others and in some cases it is not. A sorting algorithm comprises of comparison, swapping, and assignment operations. Algorithms like bubble sort and insertion sort are easy to follow and understand but considered inferior to some other algorithms because of their worst case time complexity of $O(n^2)$. This paper presents a new algorithm for sorting namely Couple Sort. It is a hybrid sorting algorithm which is influenced from quick sort and bubble sort techniques which results in a considerably lower time complexity by eliminating some useless comparisons.

Keywords- *Sorting; Swapping; Quick sort; Bubble sort*

I. INTRODUCTION

Sorting an array of numbers is one of the most basic problem and requirement of computer science. There are several fundamental and advanced algorithms for sorting an integer array, which are used according to their efficiency in the problem specific domain. Some sorting algorithms work well for lesser number of elements, while some are efficient for large number of elements. Some algorithms are good for arrays with predefined constraints. Some algorithms require straight forward programming or less number of lines to be written. Some particular algorithms work best on arrays which to large extent are already sorted [1,2]. Every sorting algorithm has its merits and demerits. They may perform well for some specific cases and under perform for some other cases. The proposed algorithm is conceptually simple to understand as well as implement. It is particularly efficient for less distorted arrays. Sorting algorithms generally use top down approach to solve a problem. For example, in Merge Sort breaking the large array into two halves and then breaking the resultant sub-arrays into further two halves each and so on until comparisons of two element pairs "couple" is left [3]. And then the final answer is build up by combining the smaller results. The idea behind the presented algorithm was to come up with an alternate bottom up kind of approach, that is, using this pair "couple" comparison from the beginning itself in an efficient way so as to save unnecessary comparisons and sort the array in each step.

II. CONCEPT BEHIND COUPLE SORT

The proposed couple sort algorithm groups the adjacent numbers in the array in pairs of two, starting from the first element and continues till either one or no element is left in the array. The hence formed pairs are compared and swapped if necessary. Else they are left as it is. The second step makes pairs starting from the second element to the end and similarly performs check and swapping operations. These two steps are iterated till the terminating condition is obtained.

Hybridization comes into picture so that the worst case complexity can be reduced without affecting the best case complexity. Before pairing steps are carried out, a middle element pivot is chosen and the elements are thrown either side of the pivot based on being smaller or larger than the pivot. This slightly changes the way partition is done in Quicksort [3]. The two steps of iteration are performed on these partitions as discussed above. If these two steps of the iteration result in zero number of swaps, the array has been sorted and hence the process is terminated. The pseudo code for the proposed algorithm is shown in Fig. 1.

III. COMPLEXITY ANALYSIS

Complexity analysis of the proposed couple Sort algorithm for best case and worst case is discussed here with the help of examples. The effect of hybridization step which is introduced to reduce the worst case complexity without affecting the best case complexity is also discussed with both the cases.

A. Best Case

In this case the array is already sorted and the terminating condition is obtained after traversing the array twice.

Example-

Given array -

1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20

Loop starts-

Pairing step 1-

(1,2), (3,4), (5,6), (7,8), (9,10), (11,12), (13,14), (15,16), (17,18), (19,20)

Number of swaps needed = 0

Pairing step 2- 1, (2,3), (4,5), (6,7), (8,9), (10,11), (12,13), (14,15), (16,17), (18,19), 20

Number of swaps needed = 0

Total number of swaps = 0 => Process Terminates

```

COUPLESORT (n, Array A of n elements with starting index
1, i, count, pivot)
INPUT: n, A, i, count
OUTPUT: A
BEGIN
    GET n
    GET A
    SET pivot:= A[n/2]
    SET j:= 1
    FOR i:= 1 to n
        IF (a[i] < pivot)
            SWAP a[i] and a[j]
            j := j + 1
    FOR i:= 1 to n
        IF (a[i] == pivot)
            SWAP a[i] and a[j]
            BREAK
    INITIALIZE count:= 1
    WHILE (count > 0)
        count:= 0
        FOR i:= 1 to n-1
            IF (A[i] > A[i+1])
                SWAP A[i], A[i+1]
                count:= count + 1
            END IF
        i:= i + 2
    END FOR
    FOR i:= 2 to n
        IF (A[i] > A[i+1])
            SWAP A[i], A[i+1]
            count:= count + 1
        END IF
        i:= i + 2
    END FOR
    END WHILE
    PRINT A
END

```

Figure 1: Pseudo code of the proposed Couple Sort

Hybridization Effect for the given array - 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20

Pivot= 10

Partition results in the same array because it is already sorted.

Pairing step 1- (1,2), (3,4), (5,6), (7,8), (9,10), (11,12), (13,14), (15,16), (17,18), (19,20)

Number of swaps needed = 0

Pairing step 2- 1, (2,3), (4,5), (6,7), (8,9), (10,11), (12,13)(14,15), (16,17), (18,19), 20

Number of swaps needed = 0

Total number of swaps = 0 => Process Terminates

In this case, i.e., the best case, both the 'for' loops run once and no swapping is needed in either of the loops, hence the count value remains 0 and the while loop is terminated resulting in termination. Hence the best case complexity is $O(n)$.

B. Worst Case

Reverse sorted array which involves every element to be moved to $(n-i)^{th}$ position from its initial i^{th} position.

Example-

Given array- 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

Iterations:

I=1

Pairing step1- (20,19), (18,17), (16,15), (14,13), (12,11), (10,9), (8,7), (6,5), (4,3), (2,1)

Swapping step1- (19,20), (17,18), (15,16), (13,14), (11,12), (9,10), (7,8), (5,6), (3,4), (1,2)

Pairing step2- 19, (20,17), (18,15), (16,13), (14,11), (12,9), (10,7), (8,5), (6,3), (4,1), 2

Swapping step2- 19, (17,20), (15,18), (13,16), (11,14), (9,12), (7,10), (5,8), (3,6), (1,4), 2

I=2

Pairing step1- (19,17), (20,15), (18,13), (16,11), (14,9), (12,7), (10,5), (8,3), (6,1), (4,2)

Swapping step1- (17,19), (15,20), (13,18), (11,16), (9,14), (7,12), (5,10), (3,8), (1,6), (2,4)

Pairing step2- 17, (19,15), (20,13), (18,11), (16,9), (14,7), (12,5), (10,3), (8,1), (6,2), 4

Swapping step2- 17, (15,19), (13,20), (11,18), (9,16), (7,14), (5,12), (3,10), (1,8), (2,6), 4

I=3

Pairing step1- (17,15), (19,13), (20,11), (18,9), (16,7), (14,5), (12,3), (10,1), (8,2), (6,4)

Swapping step1- (15,17), (13,19), (11,20), (9,18), (7,16), (5,14), (3,12), (1,10), (2,8), (4,6)

Pairing step2- 15, (17,13), (19,11), (20,9), (18,7), (16,5), (14,3), (12,1), (10,2), (8,4), 6

Swapping step2- 15, (13,17), (11,19), (9,20), (7,18), (5,16), (3,14), (1,12), (2,10), (4,8), 6

I=4

Pairing step1- (15,13), (17,11), (19,9), (20,7), (18,5), (16,3), (14,1), (12,2), (10,4), (8,6)

Swapping step1- (13,15), (11,17), (9,19), (7,20), (5,18), (3,16), (1,14), (2,12), (4,10), (6,8)

Pairing step2- 13, (15,11), (17,9), (19,7), (20,5), (18,3), (16,1), (14,2), (12,4), (10,6), 8

Swapping step2- 13, (11,15), (9,17), (7,19), (5,20), (3,18), (1,16), (2,14), (4,12), (6,10), 8

I=5

Pairing step1- (13,11), (15,9), (17,7), (19,5), (20,3), (18,1), (16,2), (14,4), (12,6), (10,8)
 Swapping step1- (11,13), (9,15), (7,17), (5,19), (3,20), (1,18), (2,16), (4,14), (6,12), (8,10)
 Pairing step2- 11, (13,9), (15,7), (17,5), (19,3), (20,1), (18,2), (16,4), (14,6), (12,8), 10
 Swapping step2- 11, (9,13), (7,15), (5,17), (3,19), (1,20), (2,18), (4,16), (6,14), (8,12), 10

I=6

Pairing step1- (11,9), (13,7), (15,5), (17,3), (19,1), (20,2), (18,4), (16,6), (14,8), (12,10)
 Swapping step1- (9,11), (7,13), (5,15), (3,17), (1,19), (2,20), (4,18), (6,16), (8,14), (10,12)
 Pairing step2- 9, (11,7), (13,5), (15,3), (17,1), (19,2), (20,4), (18,6), (16,8), (14,10), 12
 Swapping step2- 9, (7,11), (5,13), (3,15), (1,17), (2,19), (4,20), (6,18), (8,16), (10,14), 12

I=7

Pairing step1- (9,7), (11,5), (13,3), (15,1), (17,2), (19,4), (20,6), (18,8), (16,10), (14,12)
 Swapping step1- (7,9), (5,11), (3,13), (1,15), (2,17), (4,19), (6,20), (8,18), (10,16), (12,14)
 Pairing step2- 7, (9,5), (11,3), (13,1), (15,2), (17,4), (19,6), (20,8), (18,10), (16,12), 14
 Swapping step2- 7, (5,9), (3,11), (1,13), (2,15), (4,17), (6,19), (8,20), (10,18), (12,16), 14

I=8

Pairing step1- (7,5), (9,3), (11,1), (13,2), (15,4), (17,6), (19,8), (20,10), (18,12), (16,14)
 Swapping step1- (5,7), (3,9), (1,11), (2,13), (4,15), (6,17), (8,19), (10,20), (12,18), (14,16)
 Pairing step2- 5, (7,3), (9,1), (11,2), (13,4), (15,6), (17,8), (19,10), (20,12), (18,14), 16
 Swapping step2- 5, (3,7), (1,9), (2,11), (4,13), (6,15), (8,17), (10,19), (12,20), (14,18), 16

I=9

Pairing step1- (5,3), (7,1), (9,2), (11,4), (13,6), (15,8), (17,10), (19,12), (20,14), (18,16)
 Swapping step1- (3,5), (1,7), (2,9), (4,11), (6,13), (8,15), (10,17), (12,19), (14,20), (16,18)
 Pairing step2- 3, (5,1), (7,2), (9,4), (11,6), (13,8), (15,10), (17,12), (19,14), (20,16), 18
 Swapping step2- 3, (1,5), (2,7), (4,9), (6,11), (8,13), (10,15), (12,17), (14,19), (16,20), 18

I=10

Pairing step1- (3,1), (5,2), (7,4), (9,6), (11,8), (13,10), (15,12), (17,14), (19,16), (20,18)
 Swapping step1- (1,3), (2,5), (4,7), (6,9), (8,11), (10,13), (12,15), (14,17), (16,19), (18,20)
 Pairing step2- 1, (3,2), (5,4), (7,6), (9,8), (11,10), (13,12), (15,14), (17,16), (19,18), 20
 Swapping step2- 1, (2,3), (4,5), (6,7), (8,9), (10,11), (12,13), (14,15), (16,17), (18,19), 20

The array is sorted now.

Total computations: $n/2 \times (n-1)$ via $n/2$ outer iterations and $n-1$ inner iterations.

Hybridization Effect for the given array - 20, 19, 18, 17, 16, 15, 14, 13, 12, 11, 10, 9, 8, 7, 6, 5, 4, 3, 2, 1

Pivot = 11

After partition 10 9 8 7 6 5 4 3 2 1 11 19 18 17 16 15 14 13 12 20

Pairing and swapping is done in the same way as done without hybridization. The obtained sequences are shown below:

Iterations:

I=1

Result after pairing and swapping step1:
 9 10 7 8 5 6 3 4 1 2 11 19 17 18 15 16 13 14 12 20

Result after pairing and swapping step2:
 9 7 10 5 8 3 6 1 4 2 11 17 19 15 18 13 16 12 14 20

I=2

Result after pairing and swapping step1:
 7 9 5 10 3 8 1 6 2 4 11 17 15 19 13 18 12 16 14 20

Result after pairing and swapping step2:
 7 5 9 3 10 1 8 2 6 4 11 15 17 13 19 12 18 14 16 20

I=3

Result after pairing and swapping step1:
 5 7 3 9 1 10 2 8 4 6 11 15 13 17 12 19 14 18 16 20

Result after pairing and swapping step2:
 5 3 7 1 9 2 10 4 8 6 11 13 15 12 17 14 19 16 18 20

I=4

Result after pairing and swapping step1:
 3 5 1 7 2 9 4 10 6 8 11 13 12 15 14 17 16 19 18 20

Result after pairing and swapping step2:
 3 1 5 2 7 4 9 6 10 8 11 12 13 14 15 16 17 18 19 20

I=5

Result after pairing and swapping step1:
 1 3 2 5 4 7 6 9 8 10 11 12 13 14 15 16 17 18 19 20

Result after pairing and swapping step2:
 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20

The array is sorted now.

Thus, hybridization helps in reducing the worst case time complexity without affecting the best case time complexity. The final complexity remains of the order of n^2 but is never obtained because the array elements with nearby values are placed together and in their half of the array.

C. Use of pivots

The use of only one pivot is purely experimental. The algorithm has been tested on random numbers, reverse sorted numbers, and sorted numbers and the time taken by using no pivot, one pivot and multiple pivots has been obtained. The result can be seen in Table 1. (Data obtained by running modified algorithms on the same system and using the same software)

Table1: Comparison among Pivots

A. For 100000 Random numbers:
0 pivot- 24000-28000 ms, 1 pivot- 12000-12500 ms, and 3 pivots- 21000-22000 ms
B. For 100000 Reverse sorted numbers:
0 pivot- 19000-20000 ms, 1 pivot- 9000-9500 ms, and 3 pivots- 14000-15000 ms
C. For 100000 Sorted numbers:
0 pivot- 0 ms, 1 pivot- 0 ms, and 3 pivots- 17000-18000 ms

ms: relative milliseconds (The value is dependent on the speed of the host system's processor)

IV. COMPARISON WITH OTHER SORTING ALGORITHMS

The performance of the proposed couple sort algorithm is compared with bubble sort and quick sort. The reason of comparison with bubble sort is that bubble sort sorts the list in almost n^2 time and no reduction factor [1]. On the other hand couple sort does the same with a reduced factor; the data obtained is shown in Table 2. (Data obtained by running both the algorithms on the same system and using the same software)

Table 2: Comparison with Bubble Sort

A. For 100000 Random Numbers:
Bubble sort- 22000-23000 ms
Couple sort- 12000-12500 ms
B. For 100000 Reverse Sorted Numbers:
Bubble sort- 21000-23500 ms
Couple sort- 9000-9500 ms
C. For 100000 Sorted Numbers:
Bubble sort- 15000-15500 ms
Couple sort- 0 ms

ms: relative milliseconds (The value is dependent on the speed of the host system's processor)

Above data shows that couple sort provides a huge reduction in time when compared to n^2 type algorithm and takes absolutely no time in sorting an already sorted array. Which shows that the worst case n^2 is never obtained and the complexity remains way below $O(n^2)$.

Another comparison is performed with quick sort algorithm. The best case complexity of quicksort is $O(n \log n)$ which in case of the presented algorithm is $O(n)$. Furthermore, for reverse sorted arrays quicksort takes exactly $O(n^2)$, whereas for couple sort the time complexity ranges from $O(n \log n)$ to $O(n^2)$ depending on the array size, from small to large [4].

V. CONCLUSION

The proposed couple sort algorithm has a worst case complexity of less than n^2 , which results in lesser number of computations. It ranges from $O(n \log n)$ to $O(n^2)$ as the size of array increases. Also, the best case is linear time which proves that the proposed couple sort algorithm is efficient over many other algorithms which require unnecessary computations for already sorted list. In addition, it is an in-place algorithm, i.e., no extra memory space is required. The comparisons are well demonstrated which show the outperformance of the proposed sorting algorithm over other existing algorithms.

ACKNOWLEDGEMENT

I would take this opportunity to thank faculty members of CSE discipline, IIITDM Jabalpur, especially Dr. M K Bajpai for his guidance and for providing me a platform to come up with this paper. I would like to thank Miss Nidhi Gupta for helping me with the manuscript and revision. Thanks to the Almighty, my Parents, Tina, and everyone else involved in the successful completion of this paper.

REFERENCES

- [1] T.H. Cormen, E.L. Charles, R.L. Rivest, and C. Stein, Introduction to Algorithms, 3rd Edition, MIT Press and McGraw-Hill, August 2009.
- [2] E. Horowitz, S. Sahni and S. Anderson-Freed, Fundamentals of Data Structures, 2nd Edition, Silicon Press, 2005.
- [3] P. Biggar and D. Gregg, Sorting in the Presence of Branch Prediction and Caches, Technical Report, August 2005.
- [4] A.V. Aho, J.E. Hopcroft and J.D. Ullman, The Design and Analysis of Computer Algorithms, Addison-Wesley, 1979.